



Anil Seth

Extending J2EE Applications Using Grails

We look at how J2EE users can benefit from agile development methodologies by using Grails to maximise the returns from their middleware layer.

Exploring Grails is very much like exploring Rails. This is to be expected, since Grails was modelled on Rails. The obvious major difference is in the language being used; Rails uses Ruby, while Grails uses Groovy (http://en.wikipedia.org/wiki/Groovy_%28programming_language%29). Groovy (<http://groovy.codehaus.org/>) is a dynamic language like Ruby or Python. However, its syntax is close to Java's, so Java programmers should be very comfortable with it. The major advantage is that it "...seamlessly integrates with all existing Java classes and libraries." Furthermore, it "...compiles straight to Java bytecode; so you can use it anywhere you can use Java."

The advantages of Groovy with Java get carried forward to Grails—hence, Grails is a very attractive option for organisations that are already using Java applications. J2EE applications are very common in major corporations. By adapting Grails, J2EE users can gain the advantages of agile development methodologies.

But how easy would it be to enhance existing applications or code with modules written in Grails? The first possibility is that you may wish to use an existing database. However, Grails starts with the definition of domain classes, which are translated into a relational database using the object-relational mapping API, GORM, which is a thin layer on top of Hibernate.

Existing databases

If you wish to use an existing database, create the domain classes as per the existing database. This tutorial explains the possibilities: <https://www.ibm.com/developerworks/java/library/j-grails07158/>. You can work with a relational database—Hibernate or EJB3's.

An interesting solution is to use the database to create a CRUD application using Grails

Application Generator (<http://grag.sourceforge.net/>). The Amarok music player uses a MySQL database, so I decided to experiment with GRAG on the database used by Amarok. I selected a few tables (related to tracks) from the database, and created the Grails application. It just creates the domain classes. Creating the controllers resulted in a very uninteresting application, as the relationships between the various tables were not modelled.

I went back to GRAG and after selecting the tables, defined the relationship between the tables—and then generated the domain classes. The results were far more interesting, particularly with bi-directional relations. You could create an album and add tracks to it, or explore tracks and get information about the album.

The default locking mechanism is optimistic, as in the case of Hibernate. The version property assumes significance in this context. Since an existing database may not have the version column, it may be necessary to use pessimistic locking explicitly, where needed.

Views

The default layout of the scaffold is quite usable. There is an option to generate views, which will generate the Groovy Server Pages (GSP) for the *List*, *Create*, *Edit* and *Show* options. These can be modified to create customised views.

As you would expect, GSP can use custom JSP tags. GSP uses the '<% %>' syntax for variables, script code, expressions, etc, which makes it similar to JSP and RHTML. However, GSP also includes tags, like JSTL. The tags built into GSP are specified as '<g:example />' and are the preferred way to use GSP. Additional tags can be easily implemented using Groovy. This style is also similar to Zope's TAL.

For AJAX, Grails ships and uses the Prototype